Chapter 16

Attacking Application Architecture:

Web application architecture defines how different parts of an application interact. If the layers of the architecture are not properly separated, a single vulnerability can allow attackers to compromise the entire system. In shared environments where multiple applications run on the same infrastructure, a flaw in one application can affect others.

Tiered Architecture:

What is a Tiered Architecture?

A Tiered Architecture means dividing a web application into different layers (tiers) where each layer has a specific job.

Basic 3-Tier Architecture

1. Presentation Layer (User Interface)
2. Application Layer (Business Logic)
3. Data Layer (Database)

Why Multi-Tier Architecture is Useful?

1. Easier Development
2. Fewer Bugs
3. Code Reusability
4. Parallel Development
5. Easy Technology Replacement
6. Better Security

So, Tiered architecture divides a web application into separate layers such as presentation, application logic, and data storage. This separation makes complex applications easier to develop, maintain, and scale. Large enterprise systems may use many specialized layers and technologies within this structure. When properly implemented, multi-tier architectures improve development efficiency and can strengthen application security.

Attacking Tiered Architectures:

Three main types of attacks in tiered architectures:

Attack Type 1: Exploiting Trust Between Tiers: Different layers trust each other automatically.

Attack Type 2: Poor Separation Between Tiers: Sometimes the layers are not properly isolated.

Attack Type 3: Attacking the Infrastructure of Other Tiers: After compromising one layer, attackers may attack the servers or infrastructure supporting other layers.

So, if a multi-tier web application is poorly designed, security vulnerabilities may appear. Attackers can exploit trust relationships between different layers to move from one tier to another. Weak separation between tiers may allow a vulnerability in one layer to bypass protections in another. After compromising one tier, attackers may target the infrastructure of other tiers to gain full control of the application.

Exploiting Trust Relationships between Tiers:

What is a Trust Relationship between Tiers?

In a multi-tier web application, different layers interact with each other.

Typical tiers:

1. Presentation Layer – user interface (browser)
2. Application Layer – backend logic
3. Data Layer – database
4. Operating System Layer – server environment

These layers trust each other. That means one layer assumes the other layer is behaving correctly and securely. Normally this works fine when the application runs properly. But during attacks or unexpected conditions, this trust can be abused.

Why Trust Relationships Can Be Dangerous?

When one tier completely trusts another tier, it often does not perform its own security checks. So if an attacker compromises one tier, they can abuse the trust to attack other tiers. This makes the security breach much more serious.

So, basically, Different tiers of a web application often trust each other to perform security checks correctly. If attackers exploit vulnerabilities in one tier, they can abuse this trust to attack other tiers. For example, SQL injection can exploit the database's trust in the application layer. These trust relationships can also make security breaches harder to investigate because logs may only show the trusted system rather than the real attacker.

Subverting Other Tiers:

What Does "Subverting Other Tiers" Mean?

Subverting = bypassing or breaking security.

Why This Problem Happens?

This vulnerability often occurs when all tiers run on the same computer. This is common in cheap or small systems because it reduces cost.

So, basically, Application architecture often uses multiple tiers like web, application, and database. If these tiers are not properly separated, an attacker who compromises one tier can bypass the security controls of another tier. This commonly occurs when all tiers run on the same machine, such as in a LAMP server. For example, a path traversal vulnerability in the web application can allow attackers to

directly read MySQL database files. This bypasses database security and allows attackers to steal sensitive data like usernames and password hashes.

Attacking Other Tiers:

If an attacker compromises one tier, they may try to attack the other tiers as well. This is called Attacking Other Tiers.

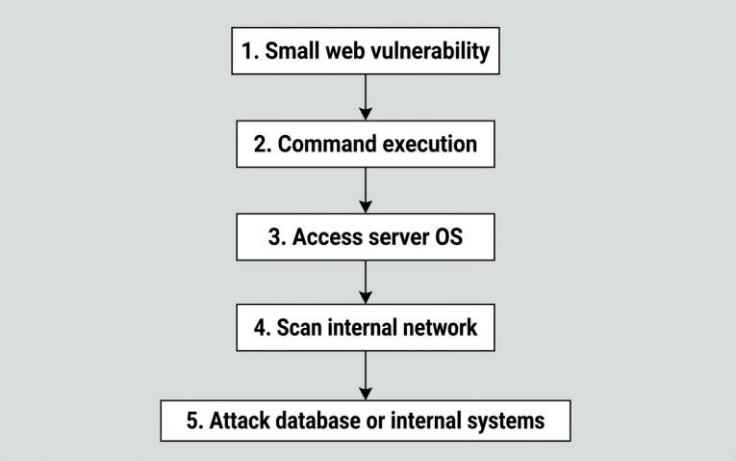
What Happens After One Server Is Compromised?

Suppose an attacker successfully hacks one server in the application. For example, they exploit a vulnerability in the web server and gain the ability to run commands on it. This is called arbitrary command execution.

Important Hacker Strategy (Hack Steps)

Security testers think creatively about vulnerabilities. Even a small vulnerability can become a major breach.

Example chain:



This is called privilege escalation and attack expansion.

So, basically, When an attacker compromises one tier of an application, they may use it to attack other tiers of the system. By executing commands on a compromised server, the attacker can scan the network, discover services, and exploit additional vulnerabilities. If the operating system of a host is compromised, all applications running on that host become vulnerable. Attackers may also move from a web application into the organization's internal network, especially through dual-homed hosts connected to multiple networks. Thus, even a small vulnerability can lead to a large infrastructure compromise.

Securing Tiered Architectures:

Tiered architecture divides an application into layers like presentation, application, and database. In a single-server LAMP setup, compromising one component can compromise the entire application. In a multi-tier setup, each layer runs separately, improving security. If one tier is attacked, the damage is limited to that tier. This isolation helps contain attacks and protect sensitive data.

Minimize Trust Relationships:

- Minimize Trust Relationships means each application tier should enforce its own security controls.
- The application server can restrict access to sensitive URLs like /admin.
- The database server can use different accounts with limited permissions for different users.
- All components should run with least system privileges.
- This approach limits damage from attacks like SQL injection, command injection, and access control flaws.

Segregate Different Components:

The principle "Segregate Different Components" means separating different parts (tiers) of an application so they cannot interact in unintended or unnecessary ways. This separation helps limit the damage if one component is compromised.

- Segregating components means separating different application tiers to prevent unintended interactions.
- Each tier should only access its own files and not files of other tiers.
- Communication between servers should be restricted to required network ports/services only.
- This helps prevent infrastructure attacks and limits the spread of compromise.
- The goal is to contain attacks and protect other parts of the system.

Apply Defense in Depth:

- Defense in Depth means using multiple layers of security in an application.
- All servers and systems should be hardened and regularly patched.
- Sensitive data like passwords and credit card numbers should be encrypted.
- Even if one security layer fails, other layers still protect the system.
- This approach helps limit the impact of attacks and protect sensitive information.

Shared Hosting and Application Service Providers:

- Shared hosting and ASPs allow organizations to host applications using third-party providers.
- Multiple customers often share the same infrastructure or servers.
- A malicious customer or vulnerable application can compromise the shared system.
- Attackers can then target other applications hosted on the same server.
- Shared servers are common targets because one compromise can affect many websites.

Virtual Hosting:

- Virtual hosting allows multiple websites to run on the same server and IP address.
- It works using the Host header in HTTP requests, which identifies the domain name.
- The server loads website files from different directories based on the domain name.
- Web servers like Apache use configurations like ServerName and DocumentRoot.
- This method is common in shared hosting environments.

Shared Application Services:

- Shared Application Services allow multiple businesses to use the same ready-made application provided by an ASP.
- Businesses customize and brand the application without building it themselves.
- This model reduces development, setup, and maintenance costs.
- It is common in industries like financial services, where many companies need similar systems.
- Since many organizations share the same platform, security issues can become complex.

Attacking Shared Environments:

- Shared environments (hosting or ASP) have multiple users on the same system, creating security risks.
- Remote access mechanisms like FTP, SCP, or direct database connections can be attacked or intercepted.
- Over-permissive access may allow users to modify other users' files or data.
- Administrative apps controlling access can have vulnerabilities that lead to privilege escalation.
- Strong segregation, encryption, and secure access controls are essential to reduce risks.

So, basically:

- In shared hosting, applications from different customers run on the same server, creating security risks.
- Malicious customers may upload backdoor scripts to execute server commands.
- Vulnerabilities like SQL injection, path traversal, and command injection in one application can compromise others.
- Shared ASP components (logs, admin panels, databases) can enable cross-application attacks.
- Weak database segregation or vulnerable shared procedures can expose data of multiple customers.

Securing Shared Environments:

- Secure shared environments by controlling customer access, segregation, and trust boundaries.
- Use strong authentication, encryption, and least-privilege access for all remote access.
- Isolate each customer's files and databases to limit impact of malicious or vulnerable code.
- Treat customized components in ASPs as untrusted and test them rigorously.
- Pay extra attention to shared logs, admin tools, and database procedures, as vulnerabilities there can affect all users.

--The End--